Déclaration sur l'honneur - absence de fonctions interdites

Conformément au §III.C du Manuel de certification SCE 2.0 ("Fonctionnalités interdites - Système de caisse enregistreuse"), le soussigné déclare sur l'honneur que le système de caisse **pili-pos v1.0.0** (build de certification, `--no-default-features --features cert`) ne comporte AUCUNE des fonctionnalités interdites suivantes :

1. Suppression d'enregistrements fiscaux

Le système **ne permet pas** la suppression d'un enregistrement fiscal déjà signé par le FDM.

Preuves :

- La table `fiscal_events` ne fait l'objet d'aucune instruction SQL `DELETE` dans le code de production.
- Protection en base : des déclencheurs SQLite (`fiscal_events_no_delete`, `fiscal_events_no_rewrite`, migration 4, `crates/pos-db/src/schema.rs`) refusent tout `DELETE` et tout `UPDATE` de cette table, quel que soit l'outil qui les émet.
- Vérification mécanique :

  ```
  grep -rn --exclude-dir=tests "DELETE FROM fiscal_events" src-tauri/ crates/
  ```

  Aucune occurrence hors modules de test : la commande (qui exclut les dossiers `tests/`) ne renvoie que deux lignes, toutes deux dans des modules `#[cfg(test)]` (`crates/pos-fiscal/src/archive.rs`, `crates/pos-db/src/migrate.rs`), où l'instruction sert à prouver que les déclencheurs la refusent. Les dossiers `tests/` contiennent le même type de preuve.
- Aucune commande `#[tauri::command]` n'expose une opération de suppression. Cf. `crates/pos-engine/src/commands/mod.rs`.

2. Modification d'enregistrements fiscaux

Le système **ne permet pas** la modification d'un enregistrement fiscal après sa signature par le FDM.

Preuves :

- La table `fiscal_events` ne fait l'objet d'aucune instruction SQL `UPDATE` dans le code de production (au-delà des écritures uniques au moment de l'INSERT initial), et les déclencheurs SQLite de la migration 4 refusent tout `UPDATE`.
- Vérification mécanique :

  ```
  grep -rn --exclude-dir=tests "UPDATE fiscal_events" src-tauri/ crates/
  ```

  Aucune occurrence hors modules de test : la commande ne renvoie que deux lignes, des tests unitaires de `crates/pos-fiscal/src/archive.rs` (`#[cfg(test)]`) qui prouvent que les déclencheurs refusent la modification.
- Le champ `archived` du schéma est en lecture seule et n'est jamais muté par l'application (vestige d'un schéma plus ancien, candidat à la suppression dans une future migration).
- La commande Tauri qui "corrige" un encaissement (`refund_checkout`, le remboursement) produit un **nouvel événement fiscal séparé** référençant l'original via `fdmRef` - la ligne originale reste intacte. Il n'existe pas de commande de correction de paiement (M132, hors périmètre v1.0.0).

3. Ajout rétroactif d'enregistrements antidatés

Le système **ne permet pas** l'ajout d'un événement fiscal portant une date/heure antérieure à l'heure courante.

Preuves :

- Le champ `posDateTime` de chaque événement est pris sur l'horloge système au moment de la construction des champs communs (`fdm_now()`, heure belge, `crates/pos-fiscal/src/event.rs`) et n'est jamais paramétré par l'utilisateur.
- Le système exige une horloge système synchronisée NTP via `chrony` (cf. `CHECKLIST_DEPLOIEMENT_KIOSK_FR.md`). L'heure ne peut pas être réglée manuellement par l'utilisateur du POS (image kiosque sans accès root, écran d'administration absent).
- Le FDM horodate lui-même chaque événement signé (`fdmDateTime`) : le SPF Finances peut comparer cette date à celle du POS.

4. Modification d'un compteur fiscal

Le système **ne permet pas** la modification arbitraire d'un compteur fiscal.

Preuves :

- Le compteur est manipulé exclusivement via les méthodes statiques `FiscalCounter::peek_next` et `FiscalCounter::next` (`crates/pos-fiscal/src/counter.rs`).
- Aucune commande Tauri n'expose d'opération `set_counter` ou équivalente.
- L'incrément est atomique avec l'archive (cf. `STRATEGIE_COMPTEURS_FR.md`) ; un compteur incrémenté ne peut pas exister sans la ligne `fiscal_events` correspondante.
- Décrément impossible : la requête `INSERT ... ON CONFLICT ... DO UPDATE current_value = +1` ne diminue jamais la valeur.

5. Désactivation programmatique du FDM

Le système **ne permet pas** de désactiver la communication avec le FDM pour effectuer des ventes "hors-fiscalité".

Preuves :

- Toute commande mutant un événement fiscal appelle `FdmClient::sign_*` ; un échec FDM en build de certification remonte une erreur bloquante (`CheckoutResult { success: false }`, message "FDM not connected - cannot complete transaction").
- Aucune option de compilation ni de configuration ne permet de contourner cette règle : le code ne contient aucun chemin qui archive ou imprime une vente non signée, dans aucune build. Les commandes en ligne reçues par HubRise ne font pas exception : chacune se termine par un événement N signé par le FDM sur ce terminal (*COMMANDES_EN_LIGNE_FR.md*). Le développement et les tests utilisent le FDM simulé (`pos-fdm-mock`). Les tests `spf_e2e_fdm_failures` vérifient, commande par commande, qu'un FDM injoignable n'archive rien et n'avance aucun compteur.
- L'URL du FDM est chargée depuis `PosConfig.fdm_url` à chaque appel via `FdmClient::new` ; il n'existe pas de variable "FDM disabled" dans `PosConfig`.

6. Effacement du journal d'événements

Le système **ne permet pas** d'effacer le journal d'événements fiscaux ni de réinitialiser la base de données via une opération utilisateur.

Preuves :

- L'image kiosque n'a pas d'écran d'administration de la base. Le binaire `sqlite3` y est installé (`deploy/image/mkosi.conf`), mais seul le wrapper `pos-db-ro` (`deploy/image/mkosi.extra/usr/local/bin/pos-db-ro`) est mis à la disposition du compte `spf` : il ouvre `pos.db` en lecture seule (`sqlite3 -readonly`). Quel que soit l'outil, la table `fiscal_events` reste protégée par les déclencheurs SQLite de la migration 4 (`crates/pos-db/src/schema.rs`), qui refusent tout `DELETE` et tout `UPDATE`.
- L'opérateur n'a pas les droits root sur la machine (image scellée).
- Une sauvegarde de la base est faite après chaque Z, une fois le Z signé et archivé (`crates/pos-engine/src/commands/signing.rs`, suite `ReportZ`, puis `report.rs::backup_database_impl`) ; elle s'écrit dans un nouveau fichier horodaté (`pos-YYYYMMDD-HHMMSS-mmm.db`). Un échec de sauvegarde

n'annule pas le Z. Le moteur garde les 7 sauvegardes les plus récentes ; il ne supprime une ancienne copie qu'après avoir vérifié la nouvelle, et jamais `pos.db`, dont la table `fiscal_events` reste en ajout seul.

7. Réimpression silencieuse d'un ticket comme s'il était neuf

Le système **ne permet pas** la réimpression d'un ticket original sans marquage "COPIE".

Preuves :

- La commande `copy_ticket` (`crates/pos-engine/src/commands/copy.rs::do_copy_ticket`) émet obligatoirement un événement C (`signCopy`) auprès du FDM, qui appose son propre `eventLabel = "C"` sur la réponse signée.
- Le ticket de copie porte en en-tête la dénomination "**COPIE**" (NL/DE "KOPIE", `crates/pos-fiscal/src/ticket.rs::ticket_type_label`), en pied la mention "CECI N'EST PAS UN TICKET DE CAISSE T.V.A. VALABLE" (`invalid_disclaimer`), et son bloc control-data porte `eventLabel: C`.
- Depuis l'ajout du ticket numérique (§III.H), cette garantie est également imposée au niveau base de données : la table `original_prints` (migration `006`) enregistre chaque impression d'un original, et `delivery::print_original_once` refuse d'imprimer un original déjà présent dans cette table (`PrintOutcome::AlreadyPrinted`). La commande de rattrapage `print_missing_original` reconstruit le ticket à partir de l'archive (une vente N ou une copie C seulement, code `print_original_unsupported` sinon) et refuse explicitement de réimprimer un original déjà sorti (code `already_printed`) ou déjà envoyé par e-mail (code `email_sent`), et renvoie vers l'écran "Copie de ticket" (événement C) dans les deux cas. L'ancienne possibilité d'une réimpression "libre" de l'original a été supprimée par cette fonctionnalité, pas seulement contournée par convention d'écran.

8. Tickets fiscaux exempts de signature ou de QR code

Le système **ne permet pas** d'imprimer un ticket fiscal exempt du bloc control-data ou du QR code.

Preuves :

- Un ticket fiscal n'est construit qu'à partir de la réponse signée du FDM (`FiscalTicket::from_sign_result` et `from_draft`, `crates/pos-fiscal/src/ticket.rs`) : aucun chemin ne l'imprime sans cette réponse. `format_receipt` (`crates/pos-print/src/receipt.rs`) imprime le bloc de données de contrôle du FDM et, dès que le FDM a renvoyé une URL de vérification, le QR Code ; une copie reprend le QR de l'original.
- Les pré-tickets (Événement P, `signPreBill`) sont visuellement distincts : dénomination "PRO FORMA" et mention "CECI N'EST PAS UN TICKET DE CAISSE T.V.A. VALABLE" (`needs_invalid_disclaimer`). Le FDM ne renvoie pas d'URL de vérification pour un événement P : ce ticket n'a pas de QR Code.

9. Multi-utilisateur sans traçabilité

Le système **ne permet pas** d'effectuer un événement fiscal sans identifier l'opérateur par son INSZ.

Preuves :

- Chaque commande Tauri reçoit un INSZ qui est inséré dans le champ `userInsz` de l'événement FDM et dans la colonne `user_insz` de la table `fiscal_events`.
- Le format de l'INSZ (11 chiffres + mod-97) est validé par `pos_core::user::validate_insz` avant tout appel commande.
- Le robot/automatisme utilise l'INSZ réservé `00000000029` (cf. spec POS v2.2 §II.3). Cette identité système ne peut pas ouvrir de session (`commands/auth.rs::do_login`) et n'apparaît pas sur l'écran de connexion.
- Un Gérant (rôle `admin`) ne peut pas se connecter sans PIN. Au tout premier démarrage seulement (aucun Gérant n'a encore de PIN), il le choisit à sa première connexion (`commands/auth.rs::do_login_with_new_pin`). Ensuite, un nouveau Gérant est créé avec son PIN, confirmé par le PIN du Gérant qui le crée (`commands/auth.rs::do_create_user`).

- Après 5 PIN erronés d'affilée, les tentatives sont refusées 30 s, durée doublée à chaque nouvel échec jusqu'à 15 min ; un redémarrage de la caisse remet les compteurs à zéro (`commands/access.rs`).

10. Données fiscales en dehors du périmètre du FDM

Le système **n'envoie jamais directement** de données fiscales au SPF Finances : toute la communication réglementaire passe par le FDM agréé, lequel transmet ensuite les données au cloud SPF Finances selon le protocole `FPSFIN_API_PROTOCOL_FDM_FINCLOUD_1.10` (2026-04-14).

Preuves :

- Vérification mécanique :

  ```
  grep -rni "finances\.fed\.be\|minfin\.fed\.be" src-tauri/ crates/ --exclude-dir=target    # →
  ```

- Le client HTTP `FdmClient` ne pointe que vers une URL configurée par l'opérateur (la FDM en LAN). Aucun client SPF Finances n'existe dans le code.

Signature de la déclaration : à apposer numériquement avec le certificat qualifié au moment de l'envoi du dossier au SPF Finances.

Distributeur : Bruno Moyse / nuagemagique.dev, Bruxelles, Belgique.

Date : à compléter à l'envoi.